

Power Management in Gerätetreibern

Runtime-PM, System-Suspend und Wakeup sicher verbinden

5.06

Drei Fragen führen durch den Energiepfad

01 Wann darf das Gerät schlafen?

Nutzung, Leerlauf und Abhängigkeiten bestimmen den zulässigen Zustand.

02 Wie bleibt der Übergang korrekt?

I/O, Registerzustand und Ressourcen brauchen eine definierte Reihenfolge.

03 Welche Evidenz zeigt die Wirkung?

PM-Status, Tracepoints, Logs und Wiederholungstests belegen den Pfad.

Lernziel

Am Ende kannst du Runtime-PM, System-Suspend und Wakeup in einem Treiber einordnen und prüfen.

Vier Zustände erklären den Runtime-PM-Zyklus

Active

Gerät ist versorgt; I/O besitzt eine aktive Nutzungsreferenz.

Idle

Keine Nutzung mehr;
Autosuspend wartet die konfigurierte Verzögerung ab.

Suspended

Treiber hat I/O beendet und entbehrliche Ressourcen abgeschaltet.

Resume

Abhängigkeiten werden hergestellt, Zustand restauriert und I/O freigegeben.

ZUSTANDSMODELL

Referenz: `get` → Zugriff → `put_autosuspend`

Übergang: `active` ↔ `suspending` ↔ `suspended`

Bedingung: kein I/O ohne aktive PM-Referenz

Vier Ebenen teilen Verantwortung im PM-Modell

PM-STRUKTUREN

```
struct dev_pm_ops
struct device
power.runtime_status
power.usage_count
PM domain
wakeup_source
PM tracepoints
```

Rollen im Modell

- 01 Runtime-PM**
Steuert ein einzelnes Gerät während des laufenden Systems.
- 02 System-Sleep**
Koordiniert Geräte beim globalen Suspend und Resume.
- 03 Wakeup**
Hält ausgewählte Ereignisquellen während des Schlafzustands wirksam.

Suspend und Resume spiegeln die Ressourcenreihenfolge

RESSOURCENPFAD

```
Resume  
Regulator einschalten  
Clock aktivieren  
Reset lösen  
Register restaurieren  
IRQ freigeben  
  
Suspend  
umgekehrte Reihenfolge
```

Drei Prüfstellen

01 Abhängigkeiten

Supplier-Geräte und PM-Domains müssen vor dem Consumer bereit sein.

02 Ruhezustand

Neue Transfers sperren und laufende Arbeit vollständig beenden.

03 Symmetrie

Nur Ressourcen abschalten, die der Callback selbst aktiviert hat.

devm_* verwaltet Lebensdauer – nicht den Zeitpunkt eines Energieübergangs.

Usage Counter und Autosuspend bilden die Runtime-Policy

RUNTIME-PM

```
usage_count  
runtime_status  
power/control  
    auto | on  
autosuspend_delay_ms  
pm_runtime_use_autosuspend()  
pm_runtime_put_autosuspend()
```

Zwei Sichtweisen

Mechanismus

Get und Put schützen die aktive Region; Callbacks ändern den Gerätezustand.

Policy

Autosuspend-Verzögerung balanciert Energiebedarf, Latenz und Transferhäufigkeit.

Ein positiver Usage Counter verhindert Runtime-Suspend – auch wenn das Gerät scheinbar idle ist.

Jeder Gerätezugriff braucht eine balancierte PM-Referenz

ZUGRIFFSCODE

```
ret = pm_runtime_resume_and_get(dev);  
if (ret < 0)  
    return ret;  
  
ret = regmap_read(st->map, REG_STATUS, &val);  
pm_runtime_put_autosuspend(dev);  
return ret;
```

Lies als Vertrag

01 Get

Das Gerät ist aktiv, bevor der erste Buszugriff beginnt.

02 Region

Alle sleep-fähigen Zugriffe liegen innerhalb derselben Referenz.

03 Put

put_autosuspend aktualisiert Last-busy, senkt den Counter und plant den Idle-Übergang.

Jeder erfolgreiche Get-Pfad benötigt genau den passenden Put-Pfad – auch bei Transferfehlern.

Runtime-Callbacks verändern Hardware in fester Reihenfolge

PM-CALLBACKS

```
runtime_suspend()  
    neue Transfers blockieren  
    IRQ-Quelle deaktivieren  
    regcache_cache_only(map, true)  
    Clock / Versorgung abschalten  
  
runtime_resume()  
    Versorgung / Clock einschalten  
    regcache_cache_only(map, false)  
    regcache_sync(map)
```

Drei Entscheidungen

01

Quiesce

Kein Callback beendet Ressourcen unter einem laufenden Transfer.

02

Zustand

Nichtflüchtige und verlorene Register getrennt behandeln.

03

Fehler

Resume-Fehler lassen das Gerät gesperrt und werden unverändert gemeldet.

System-Suspend koordiniert den gesamten Gerätebaum

SYSTEM-SLEEP

```
static const struct dev_pm_ops demo_pm_ops = {  
    .runtime_suspend = demo_runtime_suspend,  
    .runtime_resume  = demo_runtime_resume,  
    .suspend = pm_runtime_force_suspend,  
    .resume  = pm_runtime_force_resume,  
};
```

Vom Einzelgerät zum System

01 Wiederverwendung

Force-Callbacks nutzen den Runtime-PM-Pfad für System-Sleep.

02 Reihenfolge

PM-Core und Geräteabhängigkeiten ordnen Suspend und Resume.

03 Variante

Wakeup-fähige Geräte benötigen eine angepasste IRQ- und Zustandslogik.

Wakeup hält nur den notwendigen Ereignispfad aktiv

WAKEUP

```
device_init_wakeup()  
device_may_wakeup()  
enable_irq_wake()  
disable_irq_wake()  
pm_wakeup_event()  
/sys/power/wakeup_count
```

Zwei Ebenen

Arming

Nur freigegebene Geräte und IRQs werden als Wakeup-Quelle aktiviert.

Accounting

Wakeup Events verhindern oder beenden einen konkurrierenden Suspend.

Nicht jeder normale Geräteinterrupt darf das System aufwecken.

I/O, Suspend und Remove teilen denselben Zustand

NEBENLÄUFIGKEIT

I/O-Pfad

Runtime-Referenz halten

Suspend

Zugriff sperren

Work synchron stoppen

Resume

Zustand restaurieren

Remove

neue Arbeit verhindern

Runtime-PM deaktivieren

Der sichere Ablauf

01 Schützen

Lock und PM-Referenz decken den gemeinsam genutzten Zustand ab.

02 Überführen

Callbacks warten kontrolliert oder melden, dass das Gerät busy ist.

03 Entfernen

Asynchrone Arbeit stoppen, bevor PM und Ressourcen verschwinden.

Lock-Reihenfolge zwischen Bus, Treiber und PM-Pfad dokumentieren; keine Referenz unter falschem Lock anfordern.

Fehler lassen sich nach Policy, Treiber und Abhängigkeit ordnen

DIAGNOSE

```
power/runtime_status  
power/control  
power/autosuspend_delay_ms  
power/runtime_active_time  
power/runtime_suspended_time  
power/wakeup  
power tracepoints
```

Drei Fehlerklassen

- 01 Policy**
Usage Counter oder control=on verhindert den erwarteten Suspend.
- 02 Treiber**
Callback, Rückgabewert oder nicht gestoppte Arbeit blockiert den Übergang.
- 03 Abhängigkeit**
Supplier, PM-Domain oder Wakeup-Quelle hält das Gerät aktiv.

Praxis: Runtime-PM read-only beobachten

TERMINAL

```
$ dev=/sys/bus/.../devices/...  
$ cat "$dev/power/runtime_status"  
$ cat "$dev/power/control"  
$ cat "$dev/power/autosuspend_delay_ms"  
$ cat "$dev/power/runtime_suspended_time"  
$ dmesg | grep -Ei 'suspend|resume|runtime pm'
```

Belege drei Aussagen

01 Unterstützung

Vorhandene power-Dateien zeigen, welche PM-Daten exportiert werden.

02 Zustand

runtime_status und Zeitwerte beschreiben den beobachteten Übergang.

03 Ursache

Usage Counter, Policy und Log erklären, warum das Gerät aktiv bleibt.

Praxis: Den PM-Pfad eines Treibers auditieren

01 Gerät wählen

Ein Runtime-PM-fähiges Laborgerät und seinen Treiber auswählen.

02 Zugriff verfolgen

Get, Buszugriff und put_autosuspend im Datenpfad zuordnen.

03 Callbacks spiegeln

Suspend- und Resume-Reihenfolge für alle Ressourcen vergleichen.

04 Nebenläufigkeit prüfen

IRQ, Workqueue, Locks und Remove gegen den PM-Pfad abgleichen.

05 Evidenz sammeln

sysfs-Status, relevante Logs und zwei erwartete Übergänge dokumentieren.

06 Risiko benennen

Eine fehlende Referenz, falsche Reihenfolge oder Wakeup-Lücke erklären.

Read-only auditieren; keine produktiven Geräte suspendieren oder Wakeup-Quellen umkonfigurieren.

Power Management verbindet Zustand, Lebensdauer und Latenz.

Formuliere in einem Satz:

Welche Referenz und Reihenfolge beweisen einen sicheren Energieübergang?